

Modelos de Machine Learning para Análise Preditiva

MÓDULO 2 - SEMANA 2 - AULA 1

Agenda

- Árvores de Decisão
- Conceito de ensemble
- Introdução ao Random Forest
- Biblioteca Joblib

Árvores de Decisão (A Estrutura Lógica)

As Árvores de Decisão são a fundação de uma das famílias de algoritmos mais populares e poderosas do Aprendizado de Máquina. A grande beleza desse modelo é que ele funciona de maneira muito parecida com o pensamento humano na tomada de decisões lógicas.

<https://www.youtube.com/shorts/wA3YIG2I5WM>

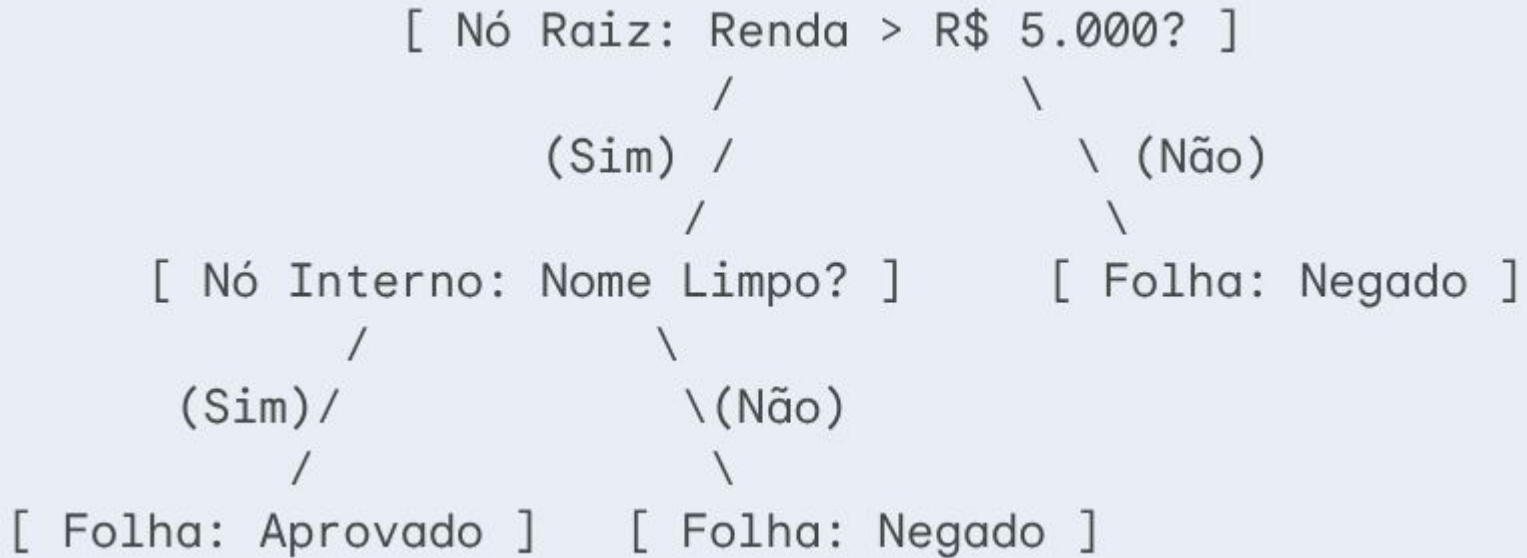
Árvores de Decisão (A Estrutura Lógica)

Pense em como um banco decide se deve aprovar um empréstimo para um cliente. O analista não faz uma conta matemática mirabolante de primeira; ele faz uma sequência de perguntas **Se-Então** (*If-Then-Else*):

- "O cliente tem nome limpo?"
 - **Não** => Empréstimo Negado.
 - **Sim** => "A renda é maior que R\$ 5.000?"
 - **Sim** => Empréstimo Aprovado.
 - **Não** => Empréstimo Negado.

Uma Árvore de Decisão traduz exatamente essa lógica visual para o computador. Ela é composta por 4 elementos fundamentais:

Árvores de Decisão (A Estrutura Lógica)



Árvores de Decisão (A Estrutura Lógica)

1. **Nó Raiz (Root Node):** É o ponto de partida absoluto da árvore. Contém todo o conjunto de dados original e faz a primeira pergunta mais importante.
2. **Nós Internos ou de Decisão (Decision Nodes):** São as perguntas intermediárias. Cada nó interno testa uma condição em uma característica (*feature*) específica dos dados.
3. **Ramos (Branches):** São as respostas às perguntas (ex: "Sim" ou "Não", "< 30" ou "> 30"). Eles conectam um nó ao próximo.
4. **Nós Folha (Leaf Nodes):** É o destino final. Um nó folha não faz mais perguntas; ele entrega a **previsão do modelo** (a classe na classificação ou um valor numérico na regressão).
- 5.

Dessa forma, vamos deixando os dados “segregados” e cada vez mais homogêneos possíveis.

Como o Algoritmo Decide Onde Cortar?

Imagine que você tem uma caixa cheia de **100 bolinhas**: 50 vermelhas e 50 azuis. Se você fechar os olhos e pegar uma bolinha, a chance de acertar a cor no chute é pura sorte. Dizemos que essa caixa está em estado de **alta impureza** (uma mistura total).

Agora imagine duas caixas organizadas:

- **Caixa A:** 50 bolinhas vermelhas e 0 azuis.
- **Caixa B:** 0 bolinhas vermelhas e 50 azuis.

A Caixa A e a Caixa B são 100% **puras**. Se você tira uma bola da Caixa A, sabe com absoluta certeza que ela é vermelha.

Como o Algoritmo Decide Onde Cortar?

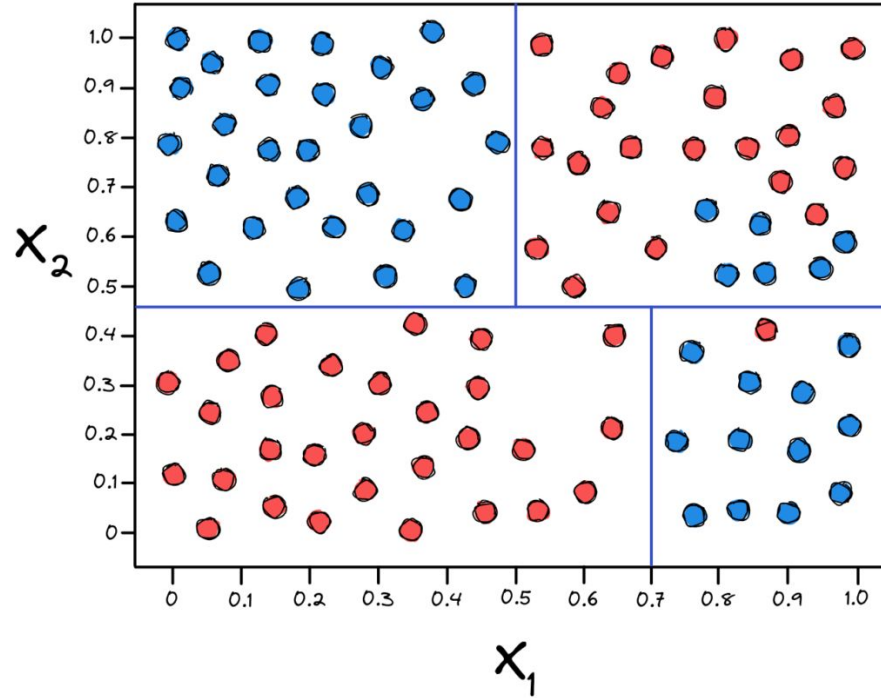
O objetivo do algoritmo da Árvore de Decisão é atuar como um organizador obsessivo: ele testa todas as variáveis disponíveis (idade, renda, histórico, etc.) e todos os pontos de corte possíveis para encontrar a divisão que gere **grupos o mais puros possível**.

- **Impureza de Gini / Entropia:** São métricas matemáticas que medem essa "bagunça". Se o nó tem apenas uma classe, a impureza é zero. Se está metade/metade, a impureza é máxima.
- **Ganho de Informação:** É a quantidade de "bagunça" que conseguimos eliminar ao fazer uma pergunta. O algoritmo escolhe sempre a pergunta que gera o **maior Ganho de Informação** no momento.

O Problema do Overfitting e a Poda (Pruning)

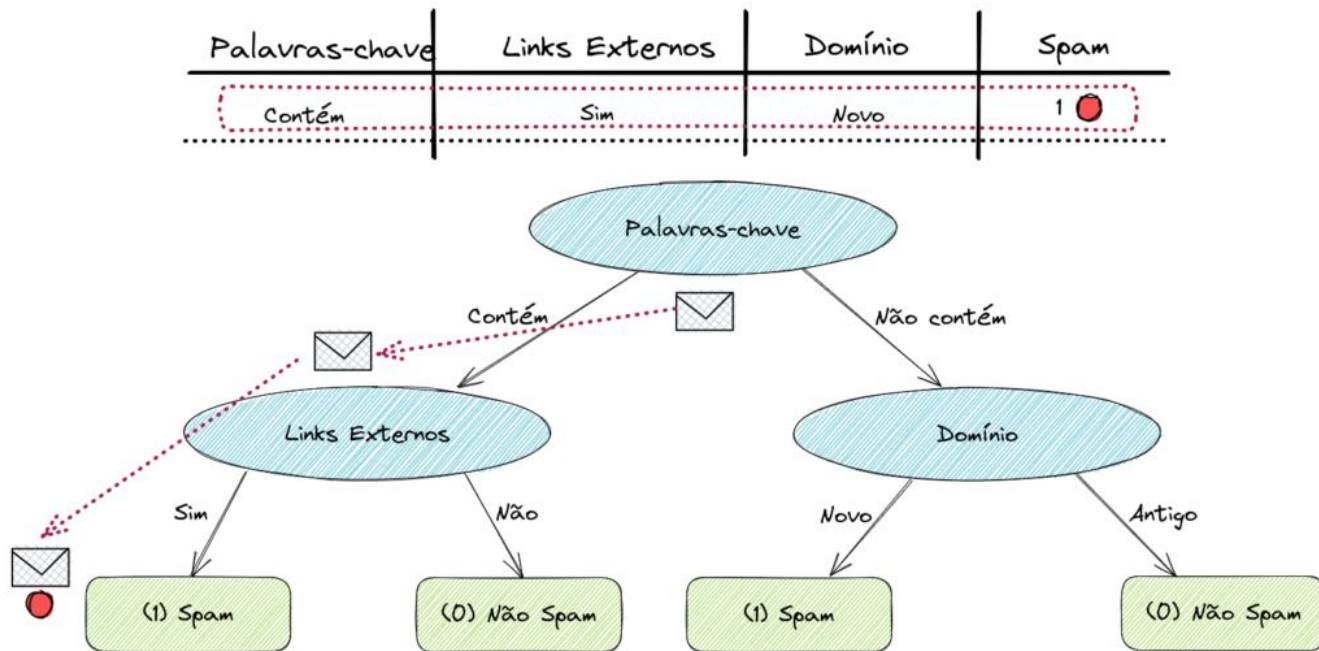
Se você não impuser limites, uma Árvore de Decisão é "gananciosa" e perfeccionista ao extremo. Ela continuará fazendo perguntas até isolar cada dado em sua própria folha individual.

Árvores de Decisão (A Estrutura Lógica)



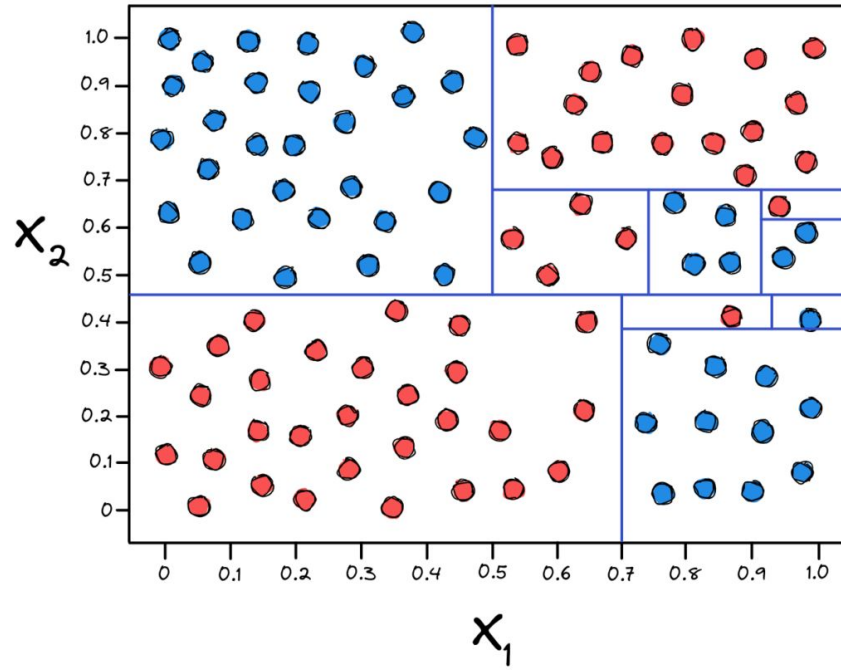
<https://brains.dev/2023/arvores-de-decisao-algoritmos-baseados-em-arvores/>

Árvores de Decisão (A Estrutura Lógica)



<https://brains.dev/2023/arvores-de-decisao-algoritmos-baseados-em-arvores/>

Árvores de Decisão (A Estrutura Lógica)



<https://brains.dev/2023/arvores-de-decisao-algoritmos-baseados-em-arvores/>

O Problema do Overfitting e a Poda (Pruning)

A Analogia do Decorador de Prova

Imagine um estudante que, em vez de entender os conceitos da matéria, decora cada vírgula, cada número de página e até a cor do papel do livro didático.

- No teste com as perguntas exatas do livro, ele tira nota **100** (erro zero no treino).
- Na prova real, com perguntas ligeiramente diferentes, ele tira **zero** (alto erro no teste).

Isso é o **Overfitting** (Sobreajuste). A árvore "decorou" os ruídos e exceções dos dados de treino e perdeu a capacidade de generalizar para novos dados.

Como Resolver? A Poda (Pruning) e Limites de Crescimento

Para evitar que a árvore crie ramificações absurdas e irrelevantes, utilizamos mecanismos de restrição:

- **Poda Prévia (Pre-Pruning):** Definimos regras de parada *antes* do treino.
 - **Profundidade Máxima (`max_depth`):** Limita o número de "níveis" de perguntas.
 - **Mínimo de Amostras para Divisão (`min_samples_split`):** Um nó só pode se dividir se tiver pelo menos N dados.
 - **Mínimo de Amostras por Folha (`min_samples_leaf`):** Uma folha precisa ter no mínimo N dados para existir.
- **Poda Posterior (Post-Pruning):** Deixamos a árvore crescer completamente e depois "cortamos" os ramos fracos que trazem pouco ganho prático.

Um pouco de Vantagens e Desvantagens

Vantagens

Alta Interpretabilidade: É possível visualizar exatamente como o modelo tomou a decisão.

Pouco Pré-processamento: Não exige normalização ou padronização de dados numéricos.

Trata Relações Não-Lineares: Capta interações complexas entre variáveis facilmente.

Desvantagens

Alta Variância (Instabilidade): Pequenas mudanças nos dados podem gerar uma árvore totalmente diferente.

Tendência Extrema ao Overfitting: Se não for regulada, decora o dataset.

Limitação de Cortes Ortogonais: Só consegue fazer divisões perpendiculares aos eixos dos dados.

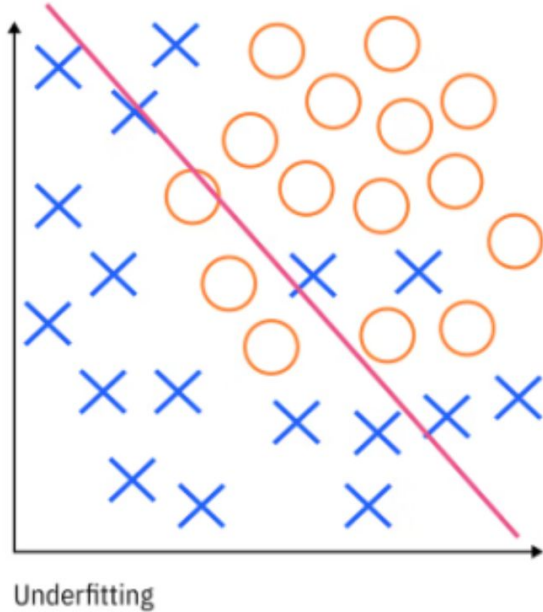
Conceito de Ensemble Learning

- Uma única Árvore de Decisão pode ser instável ou limitada. Mas o que acontece se combinarmos centenas de árvores para tomarem uma decisão juntas? Entramos no universo do Ensemble Learning (Aprendizado em Conjunto).

Conceito de Ensemble Learning

- Uma única Árvore de Decisão pode ser instável ou limitada. Mas o que acontece se combinarmos centenas de árvores para tomarem uma decisão juntas? Entramos no universo do Ensemble Learning (Aprendizado em Conjunto).

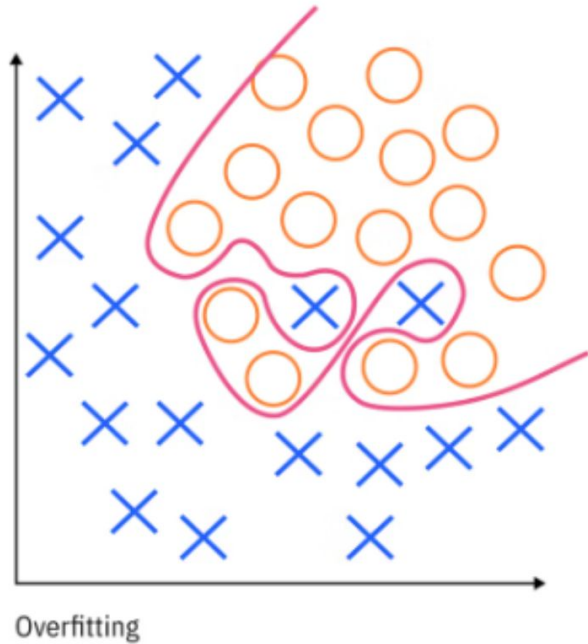
Overfitting e underfitting



Viés (Bias) - Underfitting: Ocorre quando o modelo é simplista demais. Ele ignora padrões importantes dos dados. Exemplo: Tentar prever o preço de uma casa usando apenas o número de quartos.

<https://www.ibm.com/br-pt/think/topics/overfitting-vs-underfitting>

Overfitting e underfitting



<https://www.ibm.com/br-pt/think/topics/overfitting-vs-underfitting>

Variância (Variance) - *Overfitting*: Ocorre quando o modelo é sensível demais a pequenas variações e ruídos. Ele muda drasticamente seu comportamento com pequenas alterações nos dados.

Conceito de Ensemble Learning

[Alto Viés / Baixa Variância]
(Underfitting)

0 0 0
 0 0
 0

(Longe do alvo, agrupado)

[Baixo Viés / Alta Variância]
(Overfitting)

0 0
 0 0 0
0 0

(No alvo, mas espalhado)

Estratégias de União: Bagging vs. Boosting

Bagging (Bootstrap Aggregating)

- **Como funciona:** Vários modelos são treinados de forma **independente e em paralelo**.
- **O segredo:** Cada modelo recebe uma amostra ligeiramente diferente dos dados (criada via amostragem com reposição). No final, combina-se a resposta de todos (votação ou média).
- **Foco:** Reduzir a **Variância** (evitar o overfitting).
- **Exemplo Clássico:** *Random Forest*.

Estratégias de União: Bagging vs. Boosting

Boosting

- **Como funciona:** Os modelos são treinados de forma **sequencial**.
- **O segredo:** A primeira árvore tenta prever o resultado. A segunda árvore é treinada com foco especial em **corrigir os erros cometidos pela primeira**. A terceira corrige os erros acumulados das duas primeiras, e assim por diante.
- **Foco:** Reduzir o **Viés** (tornar o modelo mais forte e preciso).
- **Exemplos Clássicos:** *XGBoost*, *LightGBM*, *AdaBoost*.

Estratégias de União: Bagging vs. Boosting

BAG-GING (Bootstrap Aggregating)

```
[Árvore 1] --\  
[Árvore 2] ---> [ Votação ]  
[Árvore 3] --/
```

(Árvores independentes e paralelas)

BOOSTING (Aprendizado Sequencial)

```
[Árvore 1]  
↓ (analisa erros da 1)  
[Árvore 2]  
↓ (analisa erros da 2)  
[Árvore 3] -> [Previsão Final]
```

Introdução ao Random Forest (A Floresta Aleatória)

O **Random Forest** é uma das aplicações mais famosas do conceito de *Bagging*. Ele resolve a instabilidade e a fragilidade de uma única árvore criando uma floresta diversificada de centenas de árvores de decisão.

3.1 O Segredo da Aleatoriedade

Se você treinar 100 árvores de decisão exatamente nos mesmos dados, todas farão as mesmas perguntas e chegarão ao mesmo resultado. Você não terá uma floresta diversificada, apenas 100 cópias idênticas da mesma árvore!

Para evitar isso, o Random Forest injeta **dois níveis de aleatoriedade**:

Introdução ao Random Forest (A Floresta Aleatória)

Bagging de Amostras (Linhas - Bootstrapping): Cada árvore da floresta é treinada com uma seleção aleatória de linhas do dataset (com reposição).

Bagging de Atributos (Colunas - Feature Subspacing): Em **cada nó de cada árvore**, o algoritmo não pode olhar todas as colunas disponíveis. Ele é forçado a escolher a melhor pergunta considerando apenas um subconjunto aleatório de variáveis (por exemplo, sorteia 3 variáveis de um total de 10).

Introdução ao Random Forest (A Floresta Aleatória)

1. AMOSTRAMENTO DE LINHAS (Bootstrap)

Dataset Original

L1	L2	L3	L4
----	----	----	----

/ | \

Tree1 Tree2 Tree3

(Cada árvore recebe um subconjunto diferente com reposição)

2. SELEÇÃO ALEATÓRIA DE COLUNAS

Nó da Árvore

Considera APENAS um subconjunto aleatório de variáveis (ex: 2 de 4) para decidir a melhor divisão.

Introdução ao Random Forest (A Floresta Aleatória)

Imagine que haja uma **variável extremamente dominante** no dataset (ex: "Renda do Cliente"). Se deixarmos, *todas* as árvores usariam essa variável como o Nó Raiz. Ao proibir que algumas árvores vejam essa variável no início, forçamos o modelo a descobrir novos padrões escondidos nas outras variáveis.

A Votação Final

Depois que todas as N árvores da floresta são treinadas de forma independente, como o Random Forest toma a decisão final?

- **Problemas de Classificação** (ex: "Fraude" ou "Não Fraude"): Ocorre o **Voto da Maioria** (*Majority Voting*). Se 80 árvores votarem "Fraude" e 20 votarem "Não Fraude", a resposta final da floresta será **Fraude**.
- **Problemas de Regressão** (ex: "Preço do Imóvel"): Calcula-se a **Média** dos palpites. Se cada árvore previr um valor numérico diferente, a previsão final será uma média ponderada de todos os palpites.

Alguns Recursos Especiais do Random Forest

Erro Out-of-Bag (OOB Score)

Como cada árvore é treinada usando apenas cerca de 63% das linhas do dataset (devido à amostragem aleatória), os 37% dos dados restantes para aquela árvore não foram vistos por ela durante o treino.

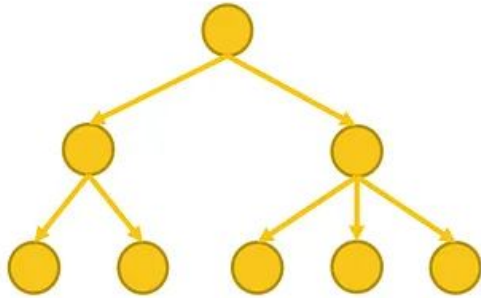
Esses dados "não vistos" são chamados de **amostras Out-of-Bag (OOB)**. O algoritmo pode usar essas amostras para testar a precisão de cada árvore automaticamente. Isso significa que o Random Forest possui um mecanismo de **validação cruzada interna e gratuita**, permitindo avaliar o modelo sem precisar separar previamente um conjunto de validação!

Importância das Variáveis (Feature Importance)

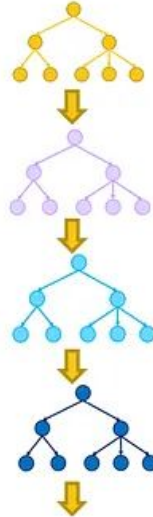
Como o Random Forest testa constantemente quais variáveis trazem maior ganho de pureza ao longo de centenas de árvores, ele **consegue calcular um ranking do impacto de cada coluna no resultado**. Isso nos dá uma visão valiosa dos dados para tomada de decisões de negócios.

Importância das Variáveis (Feature Importance)

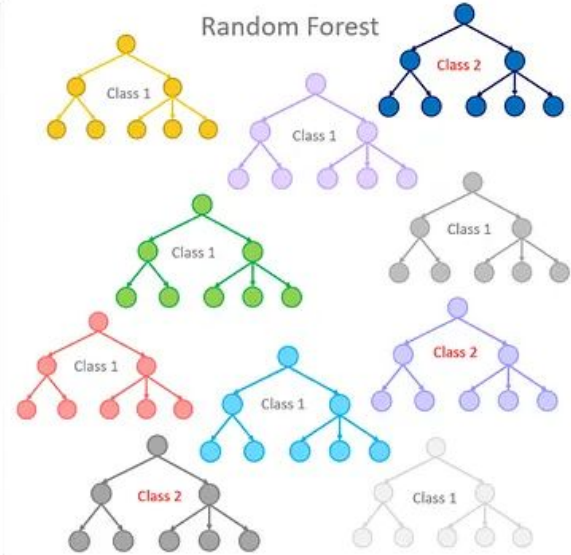
Single Decision Tree



Gradient Boosted Trees



Random Forest



<https://medium.com/analytics-vidhya/ensemble-models-bagging-boosting-c33706db0b0b>

Joblib

O Joblib é uma biblioteca Python voltada para computação científica que otimiza o processamento de dados, sendo muito utilizada para **salvar e carregar modelos de Machine Learning** de forma extremamente rápida.

Por que usá-lo para salvar modelos?

Eficiência com dados pesados: Modelos de Machine Learning guardam internamente grandes volumes de coeficientes e pesos em formato numérico. O Joblib compacta esses arrays nativamente.

Velocidade: O processo de salvar (**dump**) e ler (**load**) arquivos grandes é significativamente mais rápido que o **pickle** tradicional.

Redução de espaço: Ele possui suporte integrado para algoritmos de compressão (como **zlib**, **gzip**, **bz2** ou **lzma**).

Exemplo

```
▶ import joblib
  from sklearn.ensemble import RandomForestClassifier

  # treino do modelo
  modelo = RandomForestClassifier()
  modelo.fit(X_treino, y_treino)

  # Salvar o modelo no disco
  joblib.dump(modelo, 'meu_modelo_treinado.joblib')

  # Carregar o modelo em outro script ou servidor
  modelo_carregado = joblib.load(['meu_modelo_treinado.joblib'])
```

Quando NÃO usar o Joblib?

Segurança: Nunca carregue arquivos **.joblib** de fontes não confiáveis. Eles podem executar códigos maliciosos no seu computador durante a leitura.

Longos períodos (Produção): Arquivos Joblib podem quebrar se você atualizar a versão do Python ou da biblioteca (ex: Scikit-Learn) que criou o modelo. Para produção de longo prazo, formatos como **ONNX** ou **PMML** são mais recomendados.

Sugestão de texto sobre o assunto:

<https://medium.com/@ashaicy/serialization-and-deserialization-techniques-in-python-deserialization-69beed1ed3ef>

ATIVIDADE GUIADA

MACHINE LEARNING E PROGRAMAÇÃO | Atividade Prática de Implementação



Criada com Gemini

Conceitos complementares em Python

MÓDULO 2 - SEMANA 2 - AULA 2

Agenda

- Captura de erros com try/except
- Expressões Regulares (Regex)
- List Comprehension

Capturar de erros com try/except

O Conceito

- **Erro de Sintaxe:** O código nem executa (o Python não entende a estrutura).
- **Exceção (Erro em Tempo de Execução):** O código é válido, mas ocorre algo inesperado durante a execução (ex: dividir por zero, converter "abc" para número, abrir arquivo inexistente).

Captura de erros com try/except

- Tratar erros não é esconder falhas, é ensinar o seu programa a se recuperar graciosamente sem fechar na cara do usuário.
- Além disso, muitas vezes, essas tratativas de erros podem ser úteis para validar determinados aspectos do código e/ou inserida na lógica do programa.

Captura de erros com try/except

- Tratar erros não é esconder falhas, é ensinar o seu programa a se recuperar graciosamente sem fechar na cara do usuário.
- Além disso, muitas vezes, essas tratativas de erros podem ser úteis para validar determinados aspectos do código e/ou inserida na lógica do programa.

Exemplo sem try/except

```
# CÓDIGO SEM TRATAMENTO (quebra o programa se digitar 0 ou texto)
idade = int(input("Digite sua idade: "))
print(f"Ano que vem você terá {idade + 1} anos.")
```

Digite sua idade: Felipe

```
-----
ValueError                                Traceback (most recent call last)
/tmp/ipykernel_3232/3007898579.py in <cell line: 0>()
      1 # CÓDIGO SEM TRATAMENTO (quebra o programa se digitar 0 ou texto)
----> 2 idade = int(input("Digite sua idade: "))
      3 print(f"Ano que vem você terá {idade + 1} anos.")

ValueError: invalid literal for int() with base 10: 'Felipe'
```

Exemplo com try/except

```
# CÓDIGO PROTEGIDO
```

```
try:
```

```
    idade = int(input("Digite sua idade: "))
```

```
    print(f"Ano que vem você terá {idade + 1} anos.")
```

```
except ValueError:
```

```
    print("Erro: Por favor, digite apenas números inteiros válidos.")
```

```
Digite sua idade: Felipe
```

```
Erro: Por favor, digite apenas números inteiros válidos.
```

O Erro Comum do except: Genérico

O erro: Usar `except` sem especificar a exceção (ou usar `except Exception:`).

Por que é ruim: Ele captura **qualquer** erro indiscriminadamente, inclusive interrupções de teclado (`Ctrl + C`) ou erros de lógica que você deveria consertar, dificultando até mesmo sua depuração (*debugging*).

O except: Genérico

```
# MAU HÁBITO: Não faça isso!
try:
    resultado = 10 / int(input("Número: "))
except:
    print("Ocorreu um erro!") # Qual erro? Não sabemos!
```

```
Número: Felipe
Ocorreu um erro!
```

```
# MAU HÁBITO: Não faça isso!
try:
    resultado = 10 / int(input("Número: "))
except:
    print("Ocorreu um erro!") # Qual erro? Não sabemos!
```

```
Número: 0
Ocorreu um erro!
```

Uma boa maneira de usar

```
# BOA PRÁTICA: Trate exceções específicas!
try:
    numero = int(input("Número: "))
    resultado = 10 / numero
except ValueError:
    print("Você deve digitar um número inteiro.")
except ZeroDivisionError:
    print("Não é possível dividir por zero.")
```

```
Número: Felipe
Você deve digitar um número inteiro.
```

Uma boa maneira de usar

```
# BOA PRÁTICA: Trate exceções específicas!
try:
    numero = int(input("Número: "))
    resultado = 10 / numero
except ValueError:
    print("Você deve digitar um número inteiro.")
except ZeroDivisionError:
    print("Não é possível dividir por zero.")
```

```
Número: 0
Não é possível dividir por zero.
```


Completando o Fluxo: else e finally

try: "Tente executar este bloco."

except: "Deu erro? Execute isto."

else: "Deu tudo certo no **try**? Execute isto."

finally: "Independente do que aconteceu, **SEMPRE** execute isto ao final."

Completando o Fluxo: else e finally

```
try:
    numero = int(input("Digite um número para dividir 100: "))
    resultado = 100 / numero
except (ValueError, ZeroDivisionError) as erro:
    print(f"Não foi possível calcular. Motivo: {erro}")
else:
    # Só roda se o try for 100% bem-sucedido
    print(f"Sucesso! O resultado da divisão é {resultado:.2f}")
finally:
    # Sempre roda (excelente para fechar conexões, liberar arquivos, etc.)
    print("Operação finalizada no sistema.\n")
```

Onde Encontrar Exceções na Documentação Oficial do Python

Para aplicar o tratamento de exceções correto em Python (capturando apenas os erros esperados e evitando o uso do `except` genérico), você precisa saber quais exceções uma função, método ou operador pode disparar.

Documentação das Funções Embutidas (Built-in Functions)

Quando você usa funções nativas do Python como `int()`, `float()`, `open()`, ou `len()`, as exceções estão listadas na página oficial de **Built-in Functions**.

- **Onde consultar:** [Python Documentation - Built-in Functions](#)
- **Como identificar:** Na descrição da função, a documentação menciona diretamente o verbo **raises** (*dispara/lança*).
 - **Exemplo** na documentação do `int()`:
*"If x is not a number... raises **ValueError** if string is not a valid integer."*

Página Oficial de Exceções Nativas (Built-in Exceptions)

Para ver a lista completa de erros do ecossistema Python e a relação entre eles, existe uma página dedicada exclusivamente às exceções.

- **Onde consultar:** [Python Documentation - Built-in Exceptions](#)
- **O que você encontra lá:**
 - **Descrição detalhada:** O que causa um `ValueError`, `TypeError`, `KeyError`, `IndexError`, `ZeroDivisionError`, etc.
 - **Árvore de Hierarquia:** Mostra quais exceções herdam de outras (ex: `ZeroDivisionError` herda de `ArithmeticError`, que por sua vez herda de `Exception`).

Documentação de Módulos e Pacotes (Nativos e Externos)

Ao utilizar módulos da biblioteca padrão (como `json`, `math`, `csv`) ou pacotes terceiros (como `requests`, `pandas`), as exceções são especificadas na documentação própria do módulo.

- **Exemplo nativo (`json`):** Na doc da função `json.loads()`, indica que um JSON mal formatado dispara `json.JSONDecodeError`.
- **Exemplo de biblioteca externa (`requests`):** Na documentação do pacote `requests`, a seção *Exceptions* detalha erros como `requests.exceptions.Timeout` ou `requests.exceptions.ConnectionError`.

Consulta Direta no Terminal / IDE (Sem Abrir o Navegador)

Durante o desenvolvimento, você pode consultar a documentação de duas formas rápidas:

Método A: Função `help()` no Terminal

No REPL do Python, Jupyter Notebook ou terminal:

```
# Exibe a docstring da classe/função com a lista de argumentos e erros disparados.  
help(int)
```

Consulta Direta no Terminal / IDE (Sem Abrir o Navegador)

Método B: Inspeção pelo Traceback (Forçar o Erro em Testes)

Durante o desenvolvimento local, execute o código **sem** o bloco `try/except` com dados inválidos. A última linha do **Traceback** gerado pelo Python mostrará o nome exato da classe da exceção:

```
Traceback (most recent call last):
  File "script.py", line 2, in <module>
    resultado = 10 / 0
ZeroDivisionError: division by zero

...   File "/tmp/ipykernel_801/2688749853.py", line 1
        Traceback (most recent call last):
            ^
        SyntaxError: invalid syntax. Perhaps you forgot a comma?
```


Prompts para Consultar Exceções em Python com IA

Opção 1: Prompt Rápido (Direto ao ponto)

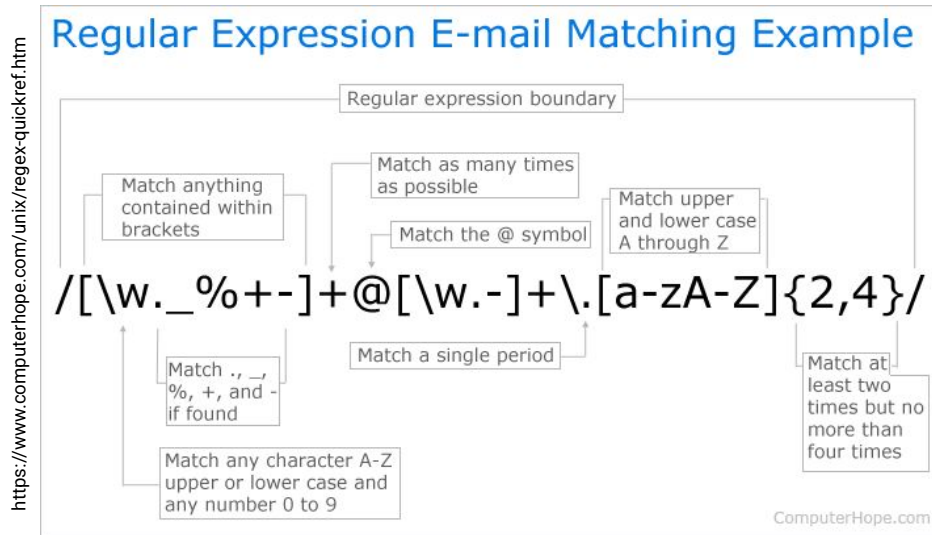
Quais exceções nativas do Python o código abaixo pode disparar ao receber dados inválidos ou falhar? Liste o nome de cada exceção e qual linha/operação a causa: (cole_seu_codigo_aqui)

Opção 2: Prompt Completo (Com estrutura `try/except` sugerida)

Atue como um tutor de Python. Analise o código a seguir e: 1. Liste as exceções específicas que podem acontecer. 2. Reestruture o código utilizando o bloco `try/except` correto para cada erro. 3. Explique sucintamente o motivo de tratar cada exceção. (cole_seu_codigo_aqui)

Introdução: O que é Regex?

Regex é a abreviação de **Expressões Regulares** (*Regular Expressions*), que consistem em sequências especiais de caracteres formando um **padrão de busca**. Elas servem para encontrar, validar ou modificar trechos de textos de forma rápida e automatizada.



O Conceito Central: Busca Normal vs. Busca com Regex

- **Busca tradicional de string** (`texto.find("123")`): Procura literalmente pelos caracteres "123". Se o texto tiver "456", ele ignora.
- **Busca com Regex**: Procura por um **formato**. Em vez de dizer *"procure por 123"*, você diz: *"procure por 3 números seguidos, não importa quais sejam"*.

A Regex é montada combinando duas coisas simples: **Quem** (o tipo de caractere) e **Quanto** (quantas vezes ele se repete).

O "QUEM" (Classes de Caracteres)

Símbolo	O que significa?	Dica de Mnemônica
\d	Qualquer dígito (0 a 9)	d de Dígito
\w	Qualquer caractere de word (letra, número ou _)	w de Word
\s	Espaço em branco (space, tab, quebra de linha)	s de Space
.	Curinga: combina com qualquer caractere	Um ponto aceita tudo
[abc]	Apenas o que estiver dentro dos colchetes	Uma "lista de opções"

O "QUANTO" (Quantificadores)

Símbolo	Quantidade	Exemplo prático
+	1 ou mais (pelo menos um deve existir)	<code>\d+</code> = "um ou mais números" (ex: 5, 42, 1000)
?	0 ou 1 (é opcional)	<code>https?</code> = o s é opcional (captura http e https)
*	0 ou mais (pode nem existir ou ter vários)	<code>a*b</code> = aceita b, ab, aab
{n}	Exatamente \$n\$ vezes	<code>\d{4}</code> = "exatamente 4 dígitos" (ex: 2026)
{min,max}	Intervalo de vezes	<code>\d{4,5}</code> = "de 4 a 5 dígitos"

Colocamos estes símbolos **logo depois** do "quem" para dizer a quantidade.

Exemplo do Telefone

Imagine que você quer capturar um telefone no formato **(11) 98765-4321**.

A Regex para isso é: `\(\d{2}\)\s?\d{4,5}-\d{4}`

Vamos quebrar essa expressão peça por peça:

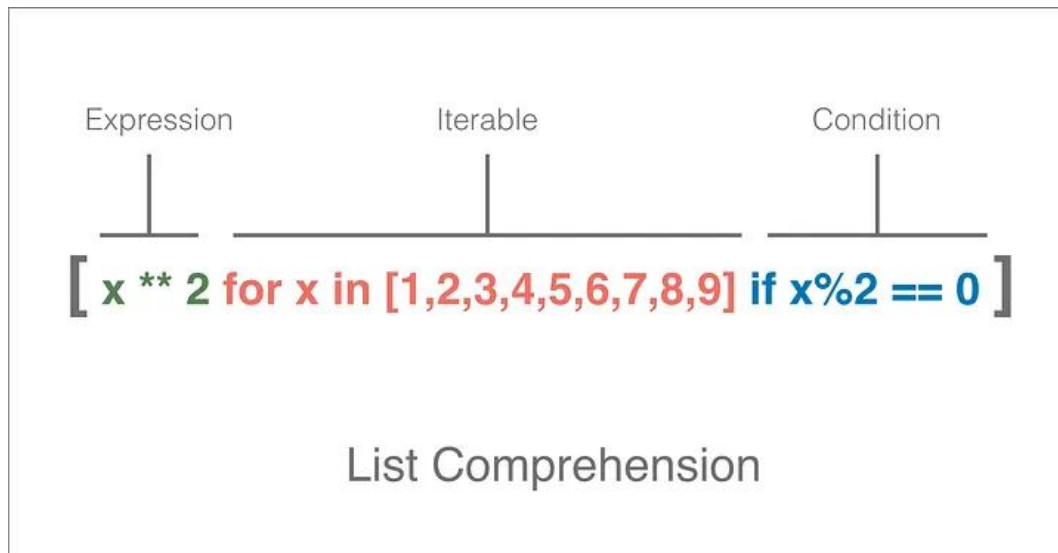
1. `\(` e `\)` : Os parênteses têm significado especial em Regex. Para procurar parênteses **literais**, colocamos a barra `\` antes.
2. `\d{2}` : Exatamente 2 dígitos (o DDD).
3. `\s?` : Pode ter 0 ou 1 espaço em branco após o DDD (opcional).
4. `\d{4,5}` : 4 ou 5 dígitos (o prefixo do telefone).
5. `-` : O hífen literal.
6. `\d{4}` : Exatamente 4 dígitos finais.

Ferramentas que podem ajudar!

- Vocês vão encontrar muitos sites e ferramentas que podem auxiliar com a montagem das regex
 - <https://regex101.com/>
 - <https://flathub.org/pt-BR/apps/me.iepure.devtoolbox>
- Usando LLMs
 - **Prompt de Exemplo:** *"Crie uma expressão regular em Python para capturar endereços de e-mail corporativos no formato usuario@empresa.com.br. Retorne apenas o padrão **re** com a explicação dos grupos."*
 - Podemos usar a LLM para entender uma expressão regular em um código:
 - **Prompt de Exemplo:** *"Explique passo a passo o que a expressão regular `^[a-zA-Z0-9._%+]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$` faz."*

List Comprehension

Assim como com o Regex, o segredo da **List Comprehension** (Compreensão de Listas) não é memorizar sintaxe, mas entender o **fluxo de leitura** que o Python faz.



<https://dataalgo.medium.com/python-list-comprehension-a-powerful-tool-for-simplifying-your-code-a65e83ea3bc6>

List Comprehension

O que é e Por que Existe?

Em Python, criar uma lista a partir de outra é uma das tarefas mais comuns.

A Abordagem Tradicional (**for** + **append**)

Tradicionalmente, para criar uma lista modificada ou filtrada, fazemos 3 passos:

1. Criamos uma lista vazia (**resultado = []**).
2. Percorremos a lista original com **for**.
3. Adicionamos o item processado com **.append()**.

List Comprehension

O Que É a List Comprehension?

É apenas um atalho sintático (conhecido como *açúcar sintático* ou *syntactic sugar*) para fazer esses mesmos 3 passos em **uma única linha**, criando a lista de forma direta e expressiva.

Detalhe que apesar dessa definição esse método é via de regra mais performático do que a estrutura do for.

Sintaxe Básica

Esqueça a ordem do Python por um segundo e pense em como nós **falamos**:

*"Eu quero [**dobrar cada número**] [**para cada número**] [**na lista de números**]."*

```
[ dobro(x)    for x in numeros ]
```

| | |

| | | 3. De onde vêm os dados (Iterável)

| | 2. Como chamamos cada item (Variável do loop)

| 1. O que fazemos com ele (Expressão/Resultado)

ATIVIDADE GUIADA

MACHINE LEARNING E PROGRAMAÇÃO | Atividade Prática de Implementação



Criada com Gemini

AVALIAÇÃO DOCENTE

O que você está achando das minhas aulas neste conteúdo?

Clique aqui ou escaneie o QRCode ao lado para avaliar minha aula.

Sinta-se à vontade para fornecer uma avaliação sempre que achar necessário.

